# Assignment: Image Processing and Neural Networks

| At a glance | |
|---|---|
| **Tasks** | Section A, image processing: Task 1 fruits. Section B, neural networks: Task 2 MLP from scratch with SGD and ADAM. Section C, Open Exploration: can a camera tell how hard a hand grips? |
| **Marks** | 35 marks: Task 1 8, Task 2 8, Open Exploration 19; 16 viva and 19 report, all team marks. Individual total = team mark × IPAC. |
| **You submit** | Report (6 pages maximum), code that runs, self-assessment form (one page, template provided), peer review on Moodle. |
| **Viva** | Week 5, on your own laptop, at most 20 minutes per team. Every student answers three questions: one on Task 1, one on Task 2, one on the Open Exploration. |
| **AI tools** | You may use AI tools for any part. The code of Tasks 1 and 2 does not have to be written by you, but you must understand it; for the Open Exploration you explain your method and your reasoning. That is what the viva marks are for. |

## Guidelines

- All deadlines are specified in Moodle, under the assessment section. Penalties will be applied for late submissions in accordance with the guidelines: https://www.ucl.ac.uk/academic-manual/chapters/chapter-4-assessment-framework-taught-programmes/section-3-module-assessment#3.12
- Please also be aware of UCL's Academic Misconduct policy: https://www.ucl.ac.uk/academic-manual/chapters/chapter-6-student-casework-framework/section-9-student-academic-misconduct-procedure. Collaboration with other teams via exchange of ideas, sharing of codes, re-using portions of the reports etc. are not allowed and will be considered as collusion.

## How You Are Assessed

- **The viva checks results and understanding.** You run the files of Tasks 1 and 2 and we look at the screen (each task says what we look for). Every student then answers three questions alone: one on Task 1, one on Task 2 and one on the Open Exploration. Every member must understand the code of Tasks 1 and 2, whoever or whichever AI tool wrote it; for the Open Exploration, every member explains the team's method and reasoning. The details are in the Viva section.

- **Your own thinking goes into the report.** Tasks 1 and 2 each name one What-if analysis; these are not examined at the viva. The Open Exploration has its own report criteria (Section C).

A What-if analysis starts with a team meeting and is written up in this order:

| Step | What you do | What goes into the report |
|---|---|---|
| **1. Predict** | Before anybody makes the change, every member says what they think will happen, and why. | One line per member, with the name: prediction and reason. The predictions do not have to agree, and a wrong one costs nothing. |
| **2. Run** | Make the change in a separate code file that prints the result, and run it together. | The name of that file, and the result as numbers or a figure. |
| **3. Explain** | Discuss why the result is what it is, and why the wrong predictions were wrong. | The explanation that the team agreed on. |

Each What-if analysis is worth 2 marks, scored like this:

| Score | The What-if analysis in the report shows |
|---|---|
| **2** | All three steps; every member's prediction is there with a reason; the result comes from the separate code file and we can reproduce it; the explanation is correct and says why the wrong predictions were wrong; within one page, with captions, axis labels and legends on the figures. Wrong predictions cost nothing. |
| **1** | Only one prediction for the whole team; or the reasons are empty ("it will get worse"); or the explanation is wrong; or we cannot reproduce the result. |
| **0** | No predictions, or no result. |

- **Code map.** Start every file with a code map: comment lines that name the sections of the file and say what each does. Use the same names as section headings in the code (lines that start with `# %%` in Python, or `%%` in MATLAB), not line numbers, so that the map stays right when the code changes. The assessor uses it to find their way around. The last line of the map names the AI model used for that file, with its version and setting, or says "none". Example:

```
# CODE MAP of Task2.py
#   Settings       learning rate, number of epochs, random seed
#   Load data      read RegressionData.csv and InitialWeights.npz
#   Forward pass   hidden outputs a1 and network output y for one sample
#   Deltas         output delta, then hidden delta
#   Update         new weights and biases (SGD)
#   Plots          fit on xtest, learning curve
#   AI use         Sol 6, High
```

```
# %% Settings
eta = 0.1;  n_epochs = 3000;  seed = 1
# %% Load data
...
```

- **Methods.** We recommend Python (from-scratch parts: pandas, NumPy and Matplotlib only); MATLAB is also accepted. The whole team uses the same language. Tasks 1 and 2: please use only methods taught in the lecture, written from scratch (in MATLAB, without the Neural Network Toolbox). For the Open Exploration, any libraries and pretrained models are allowed; this does not change the restrictions for Tasks 1 and 2.

## Section A: Locating Objects in an Image (8 marks)

In the "Assignment Files" folder, you will find an image called "Fruits.jpg" (Source: https://www.flickr.com/photos/53553325@N06/8543057650), shown in Figure 1.



Figure 1: Fruits

**Task 1: Locate Different Fruits in an Image (8 marks)**

Find the apple, the capsicum and the lemon in Fruits.jpg, each with its bounding box and its centroid. Calculate the centroid in two ways: a) the midpoint of the bounding box, and b) the moment method. The code must also work on a darker, mirrored version: put darkFactor (the dark factor: every pixel is multiplied by it, then rounded to 8 bits) and mirror (flip left-to-right) at the top of the file. It must find the three fruits for any dark factor from 0.6 to 1.0.

| Your code must | How this task is marked (8 marks) |
|---|---|
| • Draw the box and both centroids of each fruit; print both centroids of every fruit and the distance between them.<br>• No hard-coded pixel coordinates, no hand-drawn masks. | **At the viva (6 marks).** You run the file and we look at the image for dark factor 0.8 with mirroring: one box on each of the three fruits covering more than half of what is visible, nothing else boxed, both centroids in every box. Then every student answers one question about your code or the method behind it (see Viva).<br>**What-if analysis in the report (2 marks). Darkening.** Use your code and settings as submitted; do not retune them for each dark factor. A fruit is lost when it has no box, or its box covers less than half of what is visible of it. "Never lost" and "lost together" are valid results.<br>**Predict first:**<br>(1) Which fruit, if any, does your code lose first?<br>(2) At which dark factor?<br>(3) Which fruit is lost last, or is still found at 0.1?<br>**Then run:** lower the dark factor (darkFactor) from 1.0 to 0.1 in steps of 0.05. |

For Task 1 you may use: conv2 medfilt2 imgaussfilt rgb2gray find ismember sum eig. Not allowed: regionprops imfill bwperim bwlabel bwboundaries edge. The list is not exhaustive. If you are unsure, please post your question on Moodle forum.

## Section B: Neural Networks (8 marks)

**Task 2: Regression with a 1-3-1 network, from scratch (8 marks)**

Derive the backpropagation algorithm for a 1-3-1 network (1 input node, 3 hidden nodes with tanh activation, 1 linear output node) trained with stochastic gradient descent, using $E = ½ (d − y)^2$. Show the full calculation for one weight in each layer, then the general formula for the other weights of that layer. Put the derivation in an appendix (a scan of handwritten work is fine). It is required, and a viva question may ask which of its equations a line of code implements.

Write code from scratch (no neural-network library or toolbox) to train this network on the data with SGD. Then train the same network with ADAM, also written from scratch.

# ELEC0144 – Assignment

| Given | Fixed settings, so that results can be compared |
|---|---|
| `RegressionData.csv`: 41 samples, x = -1:0.05:1, `d = a*x.^3 + b*x.^2 + c*x + normrnd(0,0.02,[1,len])`<br>`InitialWeights.mat` (`.npz`): start from `W1r, b1r, W2r, b2r`<br>Test inputs: 20 equally spaced values from −0.97 to 0.93, `xtest = linspace(-0.97, 0.93, 20)` (the same call in MATLAB and in numpy) | • SGD: $\eta = 0.1$. ADAM: $\alpha = 0.005$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 1e\text{-}8$.<br>• Update after every sample, shuffle every epoch, 3000 epochs; set the random seed (the same value) at the start of every training run, so that a run gives the same result in any file.<br>• Training error = mean of $(d - y)^2$ over the 41 samples at the end of each epoch. |

| Your code must | How this task is marked (8 marks) |
|---|---|
| • Train with SGD from the given weights; plot the network output on xtest with the training data; print the final training error.<br>• Train with ADAM, again from the given weights (m, v and the step count t start at zero); plot both learning curves on one figure; print the epoch at which each first goes below 0.001, or "not reached". | **At the viva (6 marks).** You run the file and we look at the fit (does it follow the data?), the two learning curves (do both come down and flatten?) and the printed final error. Then every student answers one question about your code or the method behind it (see Viva).<br>**What-if analysis in the report (2 marks). The forgotten derivative.** In backpropagation, the delta of a hidden node is the error passed back from the output layer, multiplied by the slope of tanh at that node. That slope is $1 - a^2$, where a is the output of the node. Find this factor in your code (it may be written `1 - a.^2` or `1 - tanh(n).^2`) and replace it by 1, as if you had forgotten it.<br>**Predict first:**<br>(1) What will the fit look like?<br>(2) How will the final training error compare with the normal run?<br>**Then run:** train again with SGD and with ADAM. |

## Section C: Open Exploration — Can a camera tell how hard a hand grips? (19 marks)

Robots often learn to handle objects by watching videos of people. Such videos show the hand and the object, but not how hard the hand squeezes. The question: from camera images alone, can a program tell a light grip from a firm grip? Where the camera goes is your choice, as long as it is a setup that could be used to record demonstrations: fixed around the scene at any angle, or worn by the person (head, chest or wrist); one camera or several. Do not put a camera inside or on the object, and use no special equipment. Everything else is open: any method, library, pretrained model or public dataset is allowed; no GPU or paid AI service is needed. Your team records its own grips, then explores: try the ideas you think may work, keep what helps, drop what does not, and draw what you tried as a tree. There is no expected answer: a careful test that shows your method does not work can earn full marks.

### Your recordings

| What | Minimum |
|---|---|
| **Objects** | At least two, chosen by you, and not both soft: at least one keeps its shape when squeezed. |
| **People** | Every member of the team. |
| **Grips** | For each person and object, at least 10 light and 10 firm grips, in a random order. Write down the label before each grip. |
| **Camera** | One or more cameras (phone or laptop), placed as described above. Once chosen, keep the position, angle and lighting the same for all recordings, so that mainly the grip changes. Film no faces and nobody outside the team. |

### Three rules

| Rule | What it means |
|---|---|
| **1. Own data** | Your own recordings are part of your data; you may add other data. They are the common starting point of all teams. |
| **2. Test set** | Before you start exploring, put about 25% of your recordings aside as the test set: either part of every member's recordings, or all recordings of one member (with four members this is also 25%). The test set is never used for training. Keep whole grips together: never split the frames of one grip between sets (they look almost the same). |
| **3. Label out of the input** | No written labels, notes or scale display in what the method sees; otherwise it reads the answer. |

### Explore, and draw the tree

Try what you think may work, one idea at a time, and record it as a tree. Each node is one attempt: what you tried and why, its validation result, what you decided, and the initials of the member who did it. Keep the tree short: at least 5 and at most about 15 nodes of one line each, with at least one node by every member, showing the main attempts and decisions, not every parameter you changed. Where to stop: once the tree has at least five attempts, you may stop whenever you like. Then choose the best version so far as your final method, even if it is close to guessing: a careful test that shows a method does not work can earn full marks. An example:

```
Question: can video tell a light grip from a firm grip?
├─ Hand area (A, B): outline may change when squeezing; val 55%, near guessing → dropped
├─ Finger angles (C): fingers may bend more; hand found in only 40% of frames → dropped
└─ Pretrained features + logistic regression (D): richer than hand size; val 68% → kept
   ├─ + first-to-middle frame difference (A): squeezing may cause movement; val 72% → chosen
   ├─ Crop to the hand only (B): does the background hurt? val 66% → dropped
   └─ + a second camera from the side (C): does another view help? val 70% → dropped
Final method on the test set: 64%.
val = accuracy on validation recordings (not used for training).
Letters = who did it.
```

**Validation results.** Every number in the tree is a validation result: measured on recordings that the method was not trained on, never the training accuracy. How you split the rest of your recordings into training and validation is up to you; keep whole grips together.

### Your final method

Train every method on the training recordings only; the test set is never used for training (this includes classifier weights, thresholds and normalisation). You may run a method on the test set, look at the result and then try another method. Choose one version as your final method and submit a separate file called FinalMethod (.py or .m) that runs it on the test set and prints the result. Report that result and compare it with guessing (50% when light and firm grips are equally many).

### Going further (3 marks)

Teams that finish the above can add one extension; only one is marked. It may use your final method or a new one, and it is compared with your main result. Without an extension, the highest mark for this section is 16 of 19, and the highest total is 32 of 35.

| Extension | What you do |
|---|---|
| **A. A better reference** | The light/firm labels are what the person meant to do. Check a method against a measurement instead: does its output (for example its firm score) follow the measured force? For example, stand the object on a kitchen scale and compare the score with the reading (the scale measures pressing down, not the grip; keep the display out of the input); or use EgoTactile (huggingface.co/datasets/HustleHard/EgoTactile: video with glove pressure readings) and test whether the method ranks the firmer grips of a clip higher. |
| **B. A deeper test** | Find out where the method works and what it relies on: at least two new conditions (a new object, another camera or angle, other lighting, or their combinations) tested with your method, or ablations (for example, mask the hand or the object, remove one part of the method). |

### Work with AI

Use AI as a partner for ideas, code, debugging and writing; the code does not have to be written by you. What counts is your thinking: the question, the choices and the evidence. Name the AI model you used, with its version and setting, in the last line of the code map of each file (see Code map).

### Presentation at the viva

In about two minutes, walk through your tree and your final result (and the extension, if any), in whatever way works best. It is not marked on its own. Then each student answers one question, often about a node they did. Keep your recordings until the marks are released: the assessor may ask you to open one.

### The report

At most four pages with the six headings below; the tree goes under Method. Under the tree, one line per member says what they did. Every figure caption names the code file that produced it.

| Heading | What it contains |
|---|---|
| **Research question** | The question you want to answer, and why it is worth answering. (Part 3) |
| **Hypothesis** | What result you expect, and which result would show that you are wrong. (Part 3) |
| **Method** | The exploration tree, with one line per member under it on what they did; the final method, with a figure of its input and output. (Part 1) |
| **Validation** | How you split the recordings into training, validation and test sets; how many recordings in total (per person, object and label); the measure; how you compare with guessing. (Part 2) |
| **Results** | The final method on the test set, and the extension if you did one. (Part 2) |
| **Discussion** | What the results show and do not show, what did not work and why, what you would do next. (Part 3) |

### How the Open Exploration is marked (19 marks, team)

Viva — 4 marks (see Viva). Report — 12 marks: three parts, 4 marks each. Each part has four points, 1 mark each, checked against your code. If the "score 0" case applies, that part scores 0, whatever else is there.

| Part | Four points, 1 mark each | Score 0 |
|---|---|---|
| **1. Exploration (Method)** | **Each attempt explained.** For every attempt in the tree: why you tried it, its validation result (not the training accuracy) and what you decided.<br>**Step by step.** At least five attempts; later attempts follow from earlier results.<br>**Final method explained.** The final method is explained, with a figure of its input and output.<br>**Code matches.** The code does what the report says. | No tree or no method, or the code cannot produce the results. |
| **2. Final method (Validation, Results)** | **Test done right.** The FinalMethod file runs the final method on the test set; you say which recordings are in the test set.<br>**Data described.** How many recordings in total: per person, object and label.<br>**Compared with guessing.** The test result is compared with guessing (50% when light and firm grips are equally many).<br>**Clear tables and figures.** With captions and axis labels. | The test set was used for training; or frames of one grip were split between sets; or the label is visible in the input. |
| **3. Question and conclusions (Research question, Hypothesis, Discussion)** | **Clear question and hypothesis.** The question is clear; the hypothesis names a result that would show it wrong.<br>**Conclusions not overstated.** What the results show, and what they do not show.<br>**Failures explained.** What did not work and why, supported by results from the tree.<br>**Format and next step.** The six headings, within the page limit; a next step that follows from the results. | No research question, or no conclusions. |

Going further — 3 marks, one extension: 1 = done, with its result reported; 2 = also, the test set was not used for training, and the result is compared with your main result; 3 = also explains why the result is different and what that shows. A negative result with a valid test can earn full marks. There is no bonus for bigger models, more data or more equipment.

## Viva

- At most 20 minutes per team. The order of members is drawn by lot before the viva, and all files are open when it starts. Task 1 and Task 2 take about 6 minutes each: you run the file, then every student answers one question, in the drawn order. The Open Exploration takes about 7 minutes: a two-minute walk through the tree and the final result, then one question per student. Each question has about 1¼ minutes. Task 2 starts by minute 7 and the Open Exploration by minute 13; the viva ends at minute 20. The assessor keeps time so that every student reaches their Open Exploration question and scores what has been answered. No repair time is allocated.

| Score | Every viva question is scored like this |
|---|---|
| **2** | Correct without help, and specific to this code or this investigation. For a variable: what it holds and where it is used. For a function: what goes in and what comes out. For a line or a block: what it computes and what uses the result. For a "why" or "how" question: a reason that fits this code and its results. |
| **1** | Roughly right, but only after a hint (one hint is allowed); or vague, so that it would be true of any code ("it processes the image"); or only partly right; or answered by another member. |
| **0** | Wrong, or no answer. |

**From question scores to marks.** Every question is scored 2, 1 or 0, so the average of a part is between 0 and 2. The average is then multiplied to give the marks of that part: × 3 for Task 1 and for Task 2 (6 marks each), × 2 for the Open Exploration (4 marks). Example: in Task 1 the four students score 2, 2, 1 and 2. The average is 7 ÷ 4 = 1.75 out of 2, so the Task 1 viva mark is 1.75 × 3 = 5.25 out of 6. If all four score 2, the mark is 2 × 3 = 6. For the Open Exploration, the same scores give 1.75 × 2 = 3.5 out of 4. Marks are not rounded.

**Rules.** For Tasks 1 and 2, a question may be about a line, block, variable or function of the submitted code, or about the method behind it: why it is done this way, how a value was chosen, or what would change if it were done differently. The Open Exploration question is about the team's investigation, often about a node of the tree, not about code. If a file does not run or its result is wrong, each question of that task scores at most 1. You may prepare with AI, but no AI assistance or lookup is allowed during the viva.

**Examples.** Code: "What does this variable hold, and where is it used again?" "What goes into this function and what comes out?" "Which equation of your derivation is this line?" "How did you find this threshold?" "Why does ADAM go below 0.001 at a different epoch from SGD?" Open Exploration: "Which of your recordings did your method get most wrong, and why?" "Why did you drop this branch of your tree?"

## What to Submit

You should put everything (code AND your written report) into a zip folder, then submit the zip folder onto the submission point on Moodle. Note: please do not submit .rar file – only .zip is allowed. Only one member per team needs to submit the zip folder.

- **Report (PDF), at most 6 pages** (including text and images) using single spacing and 11 points Arial font: the two What-if analyses (up to one page each) and four pages on the Open Exploration. Not counted: cover page, references and the derivation appendix (at most two pages). Pages beyond the limits are not read. The report should have a cover page clearly indicating the following details: Report title.

Team number. Full name, student number and email address for each team member. Submission date.

- **Code.** The code will be tested! It must reproduce the reported outputs from saved data, with relative paths and documented commands. Keep Task1, Task2 and the two WhatIf_Task files (.py or .m). For the Open Exploration, include the code that produced the reported results and the FinalMethod file; it is read, not re-run. Do not submit any video. Each file starts with a code map. Code style and comments are not marked.
- **Self-assessment form (one page).** Use the template in the Assignment Files (SelfAssessment_Template.docx). One form per team: each member states in their own words what they contributed (one or two sentences), and the team together answers two questions: what could be improved in the team's solution, and what could be improved in the way the team worked.
- **References.** References should be formatted using the IEEE Citation Style. It is extremely important that all third party sources of information are properly credited and referenced in the correct manner. This includes code or text produced with an AI tool and taken over unchanged. All figures should have captions, axes labels and legends where appropriate.

## Marking Criteria

| Part | Viva | Report | Total |
|---|---|---|---|
| **Task 1: fruits** | one question per student: 6 | What-if analysis 2 | 8 |
| **Task 2: MLP, SGD and ADAM** | one question per student: 6 | What-if analysis 2 | 8 |
| **Open Exploration** | one question per student: 4 | three criteria, 0–4 each: 12; going further 3 | 19 |
| **Total** | 16 | 19 | 35 |

## Peer Review

Your team receives one mark out of 35. Individual total = team mark × your Moodle IPAC percentage (for example, 30 × 85% = 25.5 out of 35). Peer review covers attendance, effort, communication, contribution, respect, collaboration and standard of work. Peer review submission is compulsory; students who do not submit have their IPAC percentage capped at 70%.

## IPAC scores. Only give the shown percentages!

| IPAC score | Descriptor |
|---|---|
| **100%** | 1. Always actively providing suggestions, ideas, comments. Always participate in discussions. Always respectful to other students' ideas.<br>2. Excellent quality of work, can work independently and able to help others if needed.<br>3. Keep others up-to-date with the progress. Seek help early on (if needed) so as not to delay the project or create stress towards deadline. Fast in replying messages / emails. |
| **85%** | somewhere between |
| **70%** | 1. Actively providing suggestions, ideas, comments most of the time. Participate in discussions most of the time. Respectful to other students' ideas most of the time.<br>2. Works independently but quality is not very high, needs some help with work.<br>3. Slight delay in replying messages / emails. Slight delay in seeking help, thereby delaying progress slightly or create some stress towards deadline. |
| **55%** | somewhere between |
| **40%** | 1. Seldom actively providing suggestions, ideas, comments. Seldom participate in discussions. Seldom respectful to other students' ideas.<br>2. Minimal contributions, frequently needs help from others.<br>3. Huge delay in replying messages / emails. Huge delay in seeking help, thereby delaying progress badly or create huge stress towards deadline. |
| **20%** | somewhere between |
| **0%** | No participation |